

Application: are unlearned capabilities still in the weights?

The elicit-vs-teach metrics on WMDP-unlearned Zephyr-7B

Question. The main results show that the metrics separate a capability that is already in the weights (fine-tuning *elicits* it) from one that is absent (fine-tuning *teaches* it). Unlearning claims to move a model from the first case to the second. If it did, relearning is teaching and every metric should read “teach”; if the capability is only hidden, relearning is elicitation and the metrics should read “elicit”.

Setup in six lines. The original model is Zephyr-7B-beta. Four public unlearned versions of it are tested: RMU, ELM, NPO, SimNPO. The capability is WMDP-bio, 4-option multiple choice, scored as the argmax over the four answer letters (chance 0.25). The items are split at random into halves A and B. *Relearning* fine-tunes on A as text facts (question and correct answer, no options; LoRA $r=64$) and is read on the held-out half B, which it never sees. The *fine-tuning null* is the same recipe on MMLU facts, which cannot teach bio answers. There is no never-learned model, so the elicit reference is the *original* model (before training) and the *original’s own relearning* (after training); the teach reference is each metric’s own null, and the elicit/teach values of the main results are given alongside.

Reading a cell. Each cell is the measured value and the call it makes: **elicit** (reads like the original: at least half of the original’s distance from the null), *partly* (above its own null, less than half), *teach* (at its own null), — (nothing to measure: the model has no task behaviour for this metric to read). Before training, “elicit” means the capability is still present in the unlearned model.

1. Does it come back, and what brings it back?

	original	RMU	ELM	NPO	SimNPO
accuracy on B, before any training	0.67	0.31	0.35	0.26	0.42
after the unrelated fine-tune (MMLU facts)	—	0.35	0.55	0.62	0.62
after relearning on A (bio facts)	—	0.68	0.67	0.64	0.66
M17 relearning recovers B beyond the unrelated fine-tune	—	0.68 vs 0.35 elicit	0.67 vs 0.55 elicit	no room —	no room —
M17u an unrelated fine-tune alone unlocks B	—	locked: 0.31→0.35	0.35→0.55 elicit	0.26→0.62 elicit	0.42→0.62 elicit
M18 relearning cost vs the original’s (nats/token)	0	+0.15 ± 0.19 elicit	+0.03 ± 0.14 elicit	+4.6 ± 9.0 —	−0.31 ± 0.53 elicit
MMLU (general ability)	0.58	0.58	0.59	0.47	0.48

Table 1: Accuracy on the held-out half B. M17: position between the unrelated-fine-tune value and the original, 90% interval RMU 1.04 [0.88, 1.19], ELM 0.98 [0.57, 1.40]; for NPO and SimNPO the unrelated fine-tune already reaches the original, so there is no room left to measure. M17u: position between the unlearned model and the original, RMU 0.11 [−0.02, 0.23], ELM 0.64 [0.50, 0.78], NPO 0.89 [0.78, 1.00], SimNPO 0.80 [0.62, 0.99]. M18: prequential code length of relearning (each batch scored before training on it) minus the original’s, net of the same difference on the unrelated fine-tune; 0 = as cheap as the original relearning facts it knows; ± is 90%; NPO’s first steps repair a broken output, so its interval is too wide to read.

- **Every method leaves the capability in the weights:** relearning half the items brings the other half back to the original’s accuracy, and costs what the original’s own relearning costs.
- **What unlocks it differs:** for NPO and SimNPO any small fine-tune does; for RMU only bio-specific training does; ELM is in between.

2. Before any training: is the capability still readable in the unlearned model?

	metric	main results: elicit / teach	original	RMU	ELM	NPO	SimNPO
M12	hidden preference, bio (nats)	2.31 / 0.06	3.33	0.84 <i>partly</i>	0.40 <i>partly</i>	0.42 <i>partly</i>	3.52 elicit
M12	hidden preference, cyber (nats)	2.31 / 0.06	3.57	0.34 <i>partly</i>	0.13 <i>partly</i>	2.32 elicit	3.31 elicit
M16'	answer probe, bio (accuracy; shuffled ≈ 0.28)	—	0.60	0.28 teach	0.28 teach	0.47 elicit	0.49 elicit
M16'	answer probe, cyber	—	0.46	0.28 teach	0.29 teach	0.41 elicit	0.39 elicit
M9	answer depth: lens, best middle layer (nats)	2.1 / 0.0	4.88	1.18 <i>partly</i>	0.39 <i>partly</i>	0.17 <i>partly</i>	0.45 <i>partly</i>
M11	repeatable task circuit (split-half J@32)	0.60 / noise	0.64	noise teach	noise teach	noise teach	0.46 elicit
M14	answer carried by the state (swap gain)	+0.39 / +0.05	+0.66	+0.08 teach	+0.09 teach	+0.03 teach	+0.38 elicit
M15	option-reading heads (heads; flip rate)	22 / 0 heads	15; 0.73	20; 0.52 elicit	0 teach	0 teach	57; 0.70 elicit
M1★	original's 32 heads destroy U's answer (random 0.05)	—	0.93	0.99 elicit [†]	no answer —	no answer —	0.89 elicit
M10★	original's head states patched into U (accuracy)	—	0.49	0.20→0.30 <i>partly</i>	0.27→0.48 elicit	0.00→0.00 teach	0.13→0.25 <i>partly</i>
M2★	wiring, U vs original (J@256; ceiling 0.55)	0.36 / —	—	no answer —	no answer —	no answer —	0.23 <i>partly</i>
M13	accelerating descent	1.00 / 0.00	pending (block Hessian, decoder blocks 20 and 28)				
tally (elicit / partly / teach)				2 / 4 / 4	1 / 3 / 5	3 / 2 / 4	8 / 3 / 0

Table 2: Values on the unlearned model itself, no training. “Main results” gives the elicit parent and the teach parent from the arithmetic study where the metric has one. M16' adapts M16: the probe reads the correct option (4 classes), not the operands. M14: how often swapping the state at the answer position makes the model prefer the other item's answer, minus the rate with no swap. M10★: accuracy of U before and after the original's states at its 32 top heads are written in (32 random heads: RMU 0.20, ELM 0.34, SimNPO 0.16). [†] RMU's remaining answer signal is small (logit difference +0.75 vs +4.87 in the original); the original's heads carry all of it. M8 (latent reach) has no WMDP analogue and is not run.

- **SimNPO** reads like the original on 8 of 11: the circuit, the state, the heads and the probe are all intact. Only the output is pushed away from the answer.
- **NPO**: the answer is readable in the state (probe 0.47 vs original 0.60) while the output says nothing, so the damage is at the read-out.
- **ELM**: almost nothing is readable, but writing the original's head states back in turns it on (0.27 → 0.48, the original scores 0.49 on this test).
- **RMU** hides it best: probe, circuit and state swap read teach. Yet relearning brings it back in full (Table 1), so what these metrics miss in RMU is present, not absent.

3. After relearning on A: does the relearning look like eliciting or teaching?

Every relearned child is compared with the *original's* relearned child, trained with the same recipe on the same facts the original already knows; that child is the elicit reference in this setting.

	metric	main results: elicit / teach	orig. re-learned	RMU	ELM	NPO	SimNPO
M1	circuit overlap with the original, heads (J@32; chance 0.02)	0.33 / 0.02	ceiling 0.60	0.73 elicit	0.64 elicit	0.68 elicit	0.68 elicit
M1	original's 32 heads destroy the child's answer (random 0.05)	0.73 / –	0.86	0.88 elicit	0.86 elicit	0.86 elicit	0.83 elicit
M2	wiring overlap with the original (J@256; chance 0.004)	0.36 / –	ceiling 0.27	0.32 elicit	0.32 elicit	0.33 elicit	0.29 elicit
M3	head roles shared with the original (J; chance 0.01)	0.69 / 0.88	–	0.13 <i>partly</i>	0.08 teach	0.17 <i>partly</i>	0.15 <i>partly</i>
M4	circuit at step 1 vs final (J@32; ceiling)	at step 1 / mid-run	–	0.45 (0.78) <i>partly</i>	0.73 (0.73) elicit	0.49 (0.73) elicit	0.42 (0.73) <i>partly</i>
M5	gradient fades (first 1% / last 10%)	3.4× / 0.7×	3.2×	14.5× elicit	6.1× elicit	134× elicit	13.7× elicit
M6	weight write $\ \Delta W\ /\ W\ $ ($\times 10^{-3}$)	50 / 86	8.8	8.9 elicit	8.8 elicit	11.2 elicit	11.0 elicit
M7	state change: one shared push (PC1)	0.06 / 0.57	0.64	0.49 elicit	0.57 elicit	0.69 elicit	0.66 elicit
M10	child's head states patched into U (accuracy)	switches on / –	–	0.21→0.31 teach	0.33→0.54 elicit	0.00→0.00 teach	0.14→0.20 teach
tally , with Table 1 (elicit / partly / teach)				8 / 2 / 1	10 / 0 / 1	7 / 1 / 1	7 / 2 / 1

Table 3: Values on the relearned child, read on B. “Original relearned” is the elicit reference: the same training on the original, which knows the facts. M1 and M2 ceilings are the split-half overlap of the circuits themselves; overlaps above the ceiling mean the child’s circuit is as close to the original’s as the original’s is to itself. M4: overlap of the circuit after the first training step with the final one, in brackets the ceiling. M6 differs in scale from the main results because this is LoRA $r=64$ on a 7B model. M7 is judged against the original’s own relearning (0.64), not the main results’ 0.5: fine-tuning on text facts pushes one shared direction even when the facts are already known. M10: accuracy of U with the child’s states at the child’s 32 top heads written in; the child itself scores 0.54–0.57. The tally counts M17 and M18 from Table 1 (M17u is not counted: it asks what unlocks the capability, not whether it is there).

- **After relearning, all four read elicit on almost everything:** the child uses the original’s circuit (heads overlap 0.64–0.73 against a ceiling of 0.60), the original’s heads are as necessary in it as in the original’s own relearned child (0.83–0.88 vs 0.86), the wiring is the original’s, the gradient fades and the weight write is the original’s size.
- The exceptions are the same for all four and so say little about unlearning: head roles (M3) are low everywhere, and patching a child’s states into the unlearned model (M10) fails where the unlearned model’s own output is broken (NPO, SimNPO) or scrambled (RMU).

Caveats

- There is no teach anchor from this model: without a model that never learned WMDP-bio, “elicit” means “like the original”, and the teach values come from each metric’s own null and from the main results.
- The metrics before training are read on all bio items, those after relearning on half B; the first 512 items in source order score lower (original 0.51) than B (0.67), so the two blocks are not compared with each other.
- M5 and M6 read elicit for every method and so do not separate the methods; LoRA fixes the size of the weight write.
- One seed and one relearning recipe; the checkpoint with the lowest validation loss is used because held-out accuracy moves by ± 0.1 between evaluations.